实 验 报 告

姓名： 姚嫣如　　学号：21080012078
专业班级：23 网安　　课程：离散数学 II
上课老师：盛艳秀　　实验时间：2025/12/22

（一） 实验题目 ：城市交通网络中绕路方案的分析与规划

（二） 实验目的

1、掌握无向连通图生成树的求解方法和生成树个数；
2、掌握基本回路系统和环路空间的求解方法；
3、了解生成树、环路空间的实际应用。

（三） 实验要求

1、实验背景

在城市交通规划中，交通网络通常被抽象为**无向简单连通图**。例如，将城市的交通路口看作顶点，道路看作边，就构成了一个交通网络图。为了保证网络的连通性且避免资源浪费（避免冗余道路），需要找到一种**既连通又无回路**的子网络结构——“最小必要路网”（即**生成树**）。“最小必要路网”的**数量**反映了交通网络的**冗余可靠性**——生成树越多，当某些道路中断时，可供选择的替代连通方案就越多，交通网络稳健性越强。同时，识别网络中的**环路（环路空间）**对分析交通流至关重要，例如规划整个交通网络中不同出发点、不同目的地的所有绕行方案、可以用环路空间的方法系统性分解。

本实验旨在通过编程实现，从任意**给定网络的邻接矩阵**出发，系统地求解其**生成树个数**、**构建生成树**并**分析其环路空间结构。**

2、实验要求

在城市交通网络中，路口之间有道路相连，假设两个路口之间最多有一条道路（无平行边），给定该交通网络各路口之间的连接关系，求解如下问题：

$$A=\begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array}\begin{array}{c}\begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array}\\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}\end{array}$$

① 如何用图结构描述上述交通网络？（如**给出相邻矩阵**：

② 如何在交通网络中构建一个“最小必要路网”？在密密麻麻的道路网中，挑选出一套**覆盖所有路口**、但**没有任何绕行路线**的**道路集合**。这套路网像一棵树，**任意两点间只有唯一路径**。这相当于建设了一个“最小必要路网”。（提示：

构建一棵生成树，并给出生成树的相邻矩阵）

③ 计算该交通网络中有多少个不同的“最小必要路网”？在城市道路规划中，**生成树的数量**反映了交通网络的稳健性——生成树越多，当某些道路中断时，可供选择的替代连通方案就越多，交通网络稳健性越强。（提示：通过**基本关联矩阵**求**生成树个数**；求方阵的**行列式**和**秩**见参考代码。）

④ 当为一个交通网络规划绕行方案时，例如问题：当 A 点到 B 点的某条道路拥堵或封闭时，系统如何快速罗列所有不同的绕行方案？这时候可以用**环路空间的方法**将问题系统性简化，首先需要知道所有**局部区域**（例如围绕一个街区、一个公园、一个商圈）的**最小绕行单元** v1 v2 v3 v4。复杂的绕行路线，可以由这些基本单元组合而成。拥有这个“最小绕行单元库”，就拥有了构建所有可能替代路径的“乐高积木”。输出该交通网络的**“最小绕行单元库”**。（提示：**输出基本回路系统**（输出形式如：{e1e4e3, e2e5e3}））

⑤ 在拥有“最小绕行单元库”之后，这些不同的最小绕行单元的各种组合，就构成了所有不同路线中的**环路部分**，输出这些所有的不同环路，就可以规划所有的绕行方案。（提示：**输出环路空间**（输出形式如：{Φ, e1e4e3, e2e5e3, e1e4e5e2}））

（四） 实验内容和实验步骤；

一．需求分析

1、本实验旨在利用计算机编程模拟城市交通网络，将路口与道路抽象为无向简单连通图。程序需要对给定的交通网络图（即邻接矩阵）进行分析，求解其“最小必要路网”（即生成树），计算网络连通的稳健性（即生成树的总数量），并利用“环路空间”的方法系统地识别网络中的所有绕行方案（即基本回路系统和环路空间）。

2、输入形式

① 节点数量 n：一个整数，代表图顶点的个数。

② 邻接矩阵 A：一个 n x n 的二维矩阵。矩阵元素 A[i][j]表示第 i 个顶点到第 j 个顶点之间有道路相连。由于是无向图，矩阵关于主对角线对称，且对角线元素为 0。

3、输出的形式

- 图形化输出：

① 初始图：可视化显示原始城市交通网络，并标定顶点和边。

② 结果图：在图中高亮显示计算出的任意一棵生成树。

- 文本输出：

① 原始网络确认：输出输入的邻接矩阵 A，确认图结构。

② 最小必要路网（生成树）：输出构成该生成树的边集。

③ 生成树个数：输出一个整数，表示该交通网络中所有可能的不同生成树的总数（即“最小必要路网”的方案数）。

④ 最小绕行单元库（基本回路系统）：输出基于上述构建的生成树所找到的所有基本回路，输出形式为边的序列，例如：{e1e4e3, e2e5e3}

⑤ 所有绕行方案（环路空间）：输出通过基本回路组合（异或运算/对称差）生成的所有可能的环路，输出形式应包含空集 Φ及所有组合环路，例如：

4、数据约束

① 节点数 n：顶点个数 N 为正整数（建议 N≤20 以保证递归计算行列式的效率，或者根据实际内存允许范围设定）。

② 矩阵元素：仅限 0 或 1。

5、程序所能实现的功能

① 输入与数据转换模块

能够读取用户输入的邻接矩阵存储图结构，遍历矩阵上三角区域识别所有边。构建大小为 N×M（N 为顶点数，M 为边数）的关联矩阵。每一列代表一条边，连接的两个顶点在列中对应位置标记为 1（或 1 和 -1，取决于具体实现，通常无向图计数可均为 1）。

② 可视化展示

利用绘图库将抽象的矩阵数据转换为直观的几何图形，分别展示“原始交通网络”和“计算出的生成树路网”。

③ 生成树计数模块（矩阵树定理）

取关联矩阵去除任意一行后的子矩阵 B_r，计算方阵 $(B_r \times B_r^T)$ 的行列式的值，该值即为生成树的总个数。

④ 生成树构建与基本回路求解模块

利用 BFS 广度优先算法遍历图，标记树边与非树边（弦），构建一棵具体的生成树。对于每一条弦，寻找其在树中的唯一路径，形成基本回路，从而得到“最小绕行单元库”。

⑤ 环路空间生成模块

基于求得的基本回路系统，利用集合的异或运算（对称差）或模 2 加法，对基本回路的所有子集进行组合，生成并输出包含空集在内的所有可能的

环路（绕行方案）。

二、概要设计

说明用到的数据结构定义、主程序的流程及各程序模块之间的调用关系。

1、数据结构定义

- 图的矩阵表示：

  变量：A

  类型：numpy.array

  定义：一个 n×n 的二维整数数组。A[i][j] = 1 表示存在边，0 表示无边。用于图的遍历和连通性判断。矩阵关于主对角线对称，且对角线元素为 0。

  变量：M

  类型：numpy.array

  定义：一个 n×m 的二维整数数组。行对应顶点，列对应边。用于利用矩阵树定理计算生成树个数。

- 边的集合与映射：

  变量名：边列表 list

  类型：List[int] (列表)

  定义：存储元组 (u, v)，表示图中的边。

- 标签映射：

  变量名：lable

  类型：哈希表

  定义：使用 dict (哈希表) 存储边的编号映射关系，如 {(0, 1): 'e1'}，用于将内部坐标转换为实验要求的输出格式。

- 树边集合：

  变量名：tree_edges

  类型：set

  定义：使用 set 存储生成树的边，利用哈希特性实现 O(1) 的快速查找，区分树边与弦。

- 可视化辅助：

  使用 networkx.Graph 对象存储图结构，用于可视化绘制。

2. 主程序流程

(1) 输入与初始化：

调用 input_A() 获取用户输入的邻接矩阵，并验证输入格式；调用 edge_label() 建立边与 e_i 编号的映射关系。

(2) 原始网络可视化：

调用 visualize() 展示原始交通网络图。

(3) 生成树计数：

调用 get_M() 将邻接矩阵转换为关联矩阵.

调用 count_T() 利用矩阵树定理计算生成树的总数量。

(4) 构建生成树：

调用 build_tree_bfs() 使用广度优先搜索构建一棵具体的生成树，输出树的边集。

(5) 求解环路空间：

调用 get_fund_cycles()：遍历所有非树边，在树中寻找路径形成基本回路系统。

调用 cycle_space()：对基本回路进行幂集组合（异或运算），生成完整的环路空间。

(6) 结果输出与可视化：

调用 cycle_output() 格式化输出集合，最后调用 visualize() 展示计算出的最小必要路网（生成树）

3. 程序模块之间的调用关系

main (主控模块)：作为调度中心，按顺序调用各功能模块。

(1) 计算模块：

count_T 依赖 value_of_matrix（递归求行列式）。

get_M 依赖邻接矩阵输入。

(2) 图算法模块：

get_fund_cycles 依赖 path_in_tree（DFS 寻路）。

build_tree_bfs 独立运行，仅依赖邻接矩阵。

cycle_space 依赖 get_fund_cycles 的输出结果。

(3) 辅助模块：

visualize 和 cycle_output 依赖 edge_label 生成的标签字典。

三、详细设计

```
import numpy as np   # 处理矩阵数组
import matplotlib
matplotlib.use('TkAgg') # 或者 'Qt5Agg'，强制指定后端
```

```
import matplotlib.pyplot as plt    # 画图展示
import networkx as nx    # 辅助画图

# 获取邻接矩阵
def input_A():
    try:
        n = int(input("请输入顶点数 n："))
        print(f"请逐行输入邻接矩阵 (共 {n} 行，每行 {n} 个数字，空格分隔):")

        # while True:
        A = []
        for i in range(n):
            row_str = input(f"第 {i + 1} 行: ").split()    # 按照空格来分割字符
            row_int = [int(x) for x in row_str]      # 转换为 int
            # 输入判断
            if len(row_int) != n:
                print("该行输入错误!! 请重新输入！")
                i -= 1
                continue
            A.append(row_int)

        return np.array(A),n
    except ValueError:
        print("输入格式错误，请重新输入！")
        exit()

# 为图中的每一条边分配编号 (e1, e2, e3...)
def edge_label(n, A):
    label = {}
    count = 1

    for i in range(n):
        for j in range(i + 1, n):     # 上三角
            if A[i][j] != 0:
```

```
                    # 规范化边 (小, 大)
                    edge_tuple = tuple(sorted((i, j)))
                    num = f"e{count}"
                    label[edge_tuple] = num
                    print(f"    {num}: ({i}, {j})") # 打印对照关系
                    count += 1
    return label

# 将环路列表转换为格式字符串。
def cycle_output(cycles_list, label):
    output = []

    for cycle in cycles_list:
        if not cycle:   # 如果是空集
            output.append("Φ")
            continue

        # 把环里的每一条边转换为标签 (e1, e2...)
        elabel = []
        for edge in cycle:
            norm_edge = tuple(sorted(edge))
            if norm_edge in label:
                elabel.append(label[norm_edge])

        # 拼接成字符串 "e1e2e3"
        cycle_str = "".join(elabel)
        output.append(cycle_str)

    # 4. 最终用大括号包围
    return "{" + ", ".join(output) + "}"

# 转换为关联矩阵
def get_M(A,n):
    edges = []
    # 提取所有边 (只遍历上三角，避免重复)
```

```
    for i in range(n):
        for j in range(i + 1, n):
            if A[i][j] != 0:
                edges.append((i, j))

    m = len(edges)

    # 初始化关联矩阵 M: 行=顶点，列=边
    M = np.zeros((n,m), dtype=int)

    # 构建"有向"关联矩阵,方便利用矩阵树定理
    for i, (u, v) in enumerate(edges):
        M[u][i] = 1              # 起点为 1，终点为-1
        M[v][i] = -1

    return M, edges, m

# 递归求行列式的值
def value_of_matrix(m):
    n = len(m)
    if n == 1:
        return m[0][0]
    if n == 2:
        return m[0][0] * m[1][1] - m[0][1] * m[1][0]

    sum = 0
    # 沿第一行展开
    for k in range(n):
        if m[0][k] == 0:
            continue
        # 余子式矩阵
        sub_m = []
        for i in range(1, n):     # 除第一行
            row = []
            for j in range(n):
```

```
                    if j == k:
                        continue
                    row.append(m[i][j])
                sub_m.append(row)

            # 递归计算
            if k % 2 == 0:
                sum += m[0][k] * value_of_matrix(sub_m)
            else:
                sum -= m[0][k] * value_of_matrix(sub_m)

        return sum

# 利用关联矩阵计算生成树个数
# 生成树个数 = det(B_r * B_r.T)
def count_T(M):
    # Br 去除任意一行的余子式
    B_r = M[:-1, :]          # 去除最后一行

    # 计算矩阵乘积  B_r * B_r^T
    mul = np.dot(B_r, B_r.T)

    # 计算行列式
    # 转换成列表传给递归函数
    mul_list = mul.tolist()
    value = value_of_matrix(mul_list)

    return value

# 使用 bfs 广度构建生成树
def build_tree_bfs(n, A):
    tree_A = np.zeros((n, n), dtype=int)
    tree_edges = []
    visited = [False] * n
```

```
    # 从节点 0 开始
    queue = [0]
    visited[0] = True

    while queue:
        current = queue.pop(0)
        for v in range(n):
            # 如果 current 和 v 有边，且 v 未访问
            if A[current][v] == 1 and not visited[v]:
                visited[v] = True
                queue.append(v)

                tree_A[current][v] = tree_A[v][current] = 1
                #  (小, 大) 方便比较
                if current < v:
                    tree_edges.append((current, v))
                else:
                    tree_edges.append((v, current))

    return tree_A, tree_edges        # 树的邻接矩阵，树的边集合

# 在树中找路径
# 两点间的唯一路径
def path_in_tree(tree, start, end, path, visited):
    visited[start] = True
    path.append(start)

    if start == end:
        return True

    for node in range(len(tree)):
        if tree[start][node] == 1 and not visited[node]:
            if path_in_tree(tree, node, end, path, visited): # dfs
                return True
```

```
        path.pop()    # 回溯
        return False

# 求基本回路系统（最小绕行单元库）
# 遍历所有弦，每条弦加到树上形成唯一的环。
def get_fund_cycles(n, total_edges, tree_edges, tree_A):
        fund_cycles = []

        tree = set(tree_edges)              # 列表转为 set 方便运算

        # 找出弦 = 所有边 - 树边
        for (u, v) in total_edges:
            if (u, v) not in tree and (v, u) not in tree:            # 弦 (u, v)
                # 在树中找 u -> v 的路径
                path = []
                visited = [False] * n
                path_in_tree(tree_A, u, v, path, visited)

                cycle = []          # 表示为边的集合
                # 路径 + 弦 = 环
                for i in range(len(path) - 1):
                    p1, p2 = path[i], path[i + 1]
                    edge = tuple(sorted((p1, p2)))    # 将 sorted 返回的列表转化
为元组
                    cycle.append(edge)          # 路径 -> 环
                # 添加弦
                cycle.append(tuple(sorted((u, v))))

                fund_cycles.append(sorted(cycle))

        return fund_cycles

# 求所有绕行方案（环路空间）
# 对基本回路的所有子集进行异或运算。
def cycle_space(fund_cycles):
```

```
    cycles = []
    # 0 到 n 的组合
    import itertools
    for i in range(len(fund_cycles) + 1):
        for combine in itertools.combinations(fund_cycles, i):      # 组合基本回路
            # 对 combine 中的所有环进行边集的异或运算
            count = {}
            for cycle in combine:
                for edge in cycle:
                    count[edge] = count.get(edge, 0) + 1      # 统计每条边在选中的那些回路中总共出现了多少次

            # 边出现奇数次保留，偶数次抵消
            c = [edge for edge, n in count.items() if n % 2 == 1]
            cycles.append(sorted(c))

    return cycles

# 可视化
def visualize(A,flag,tree,label):
    n = len(A)

    # 创建图对象
    G = nx.Graph()
    for i in range(n):
        G.add_node(i)
        for j in range(i + 1, n):
            if A[i][j] == 1:
                G.add_edge(i, j)

    # 布局
    pos = nx.spring_layout(G, seed=42)   # 固定种子保证两图位置一致
    plt.figure(figsize=(9,6))
```

```
    if flag == 1:     # 交通网络
        # plt.subplot(1, 2, 1)
        plt.title("origin transportation network")
        nx.draw(G, pos, with_labels=True, node_color='lightblue',
edge_color='gray',
                node_size=600, font_weight='bold')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=label,
                                     font_size=10, rotate=False)
    elif flag == 2:        # 生成树
        # 创建生成树对象
        T = nx.Graph()
        for i in range(n):
            T.add_node(i)
            for j in range(i + 1, n):
                if tree[i][j] == 1:
                    T.add_edge(i, j)

        # plt.subplot(1, 2, 2)
        plt.title("minimum necessary road network (spanning tree)")
        # 先画淡色的底图
        nx.draw(G, pos, with_labels=True, node_color='lightgray',
edge_color='lightgray',
                style='dashed', node_size=600)
        # 再画深色的树
        nx.draw(T, pos, with_labels=True, node_color='lightblue',
edge_color='red',
                width=2, node_size=500,font_weight='bold')

        nx.draw_networkx_edge_labels(G, pos, edge_labels=label,
                                     font_size=10, rotate=False)

    plt.tight_layout()  # 自动调整间距
    plt.show()
```

```
# 主程序
def main():
    print("========= 城市交通网络分析 =========")

    A, n = input_A()
    print(f"原始交通网络邻接矩阵 ({n}x{n}):")
    print(A)

    print("标定边：")
    label = edge_label(n, A)      # 生成边的编号 (e1, e2...)
    visualize(A, 1, A, label)

    # 生成关联矩阵 计算生成树个数
    print("\n========= 生成树个数 =========")
    M, M_edges, m = get_M(A,n)
    print(f"关联矩阵为 ({n}x{m}):")
    print(M)

    num_T = count_T(M)
    print(f"生成树个数为 : {num_T}")

    # 构建任意一棵生成树
    print("\n========= 最小必要路网（生成树） =========")
    tree_A, tree_edges = build_tree_bfs(n, A)
    visualize(A,2,tree_A, label)

    print(f"构建的 '最小必要路网' (生成树) 边集为:")
    list = [label[tuple(sorted(e))] for e in tree_edges]
    # list.sort(key=lambda x: int(x[1:]))
    print(f"{{ {', '.join(list)} }}")

    # 基本回路系统
    fund_cycles = get_fund_cycles(n, M_edges, tree_edges, tree_A)
    print(f"最小绕行单元库 (基本回路系统) - 共 {len(fund_cycles)} 个:")
    format_fund = cycle_output(fund_cycles, label)
```

```
    print(format_fund)

    # 环路空间
    print("\n========= 所有绕行方案（环路空间） =========")
    space = cycle_space(fund_cycles)
    print(f"所有绕行方案 (环路空间) - 共 {len(space)} 个:")
    space = cycle_output(space, label)
    print(space)

    visualize(A, 2, tree_A, label)

if __name__ == "__main__":
    main()
```

四、调试分析

1、调试过程中所遇到的问题及解决方法；

（1）问题：图形化界面无法显示

- 现象：程序运行结束后，命令行提示完成，但没有弹出图形窗口。
- 解决方法：这是由于 Matplotlib 后端配置问题。在代码头部添加了 matplotlib.use('TkAgg') 强制指定后端，并确保在绘图代码末尾调用 plt.show()，成功解决了弹窗问题。

（2）问题：边集合输出问题

- 现象：在输出生成树的边集以及基本回路系统时，需要输出形式如：{e1e4e3, e2e5e3}，但树边集合在存储时形式如：{（0，1），（2，4）...}。
- 解决方法：通过 edge_label 函数为每一条边标定，并以 dict (哈希表) 存储边的编号映射关系于 label 中，如 {(0, 1): 'e1'}，在输出时通过 cycle_output 函数调用 label，通过映射关系输出相应的边标定。

2、算法的时空分析；

（1） 生成树计数算法 (递归求行列式)：

- 时间复杂度：该函数采用代数余子式展开，其复杂度为阶乘级 O(N!) 当顶点数 N>10 时，计算时间会急剧增加。
- 空间复杂度：主要消耗在递归栈上，深度为 N，复杂度为 O(N)。

（2）构建生成树 (BFS)：

- 时间复杂度：广度优先搜索，复杂度为 O(V+E)，其中 V 是顶点数，E 是

边数。

- 空间复杂度：需要队列和访问标记数组，复杂度为 $O(V)$。

（3）环路空间生成：

- 时间复杂度：设基本回路个数为 $k=E-V+1$。程序使用了 itertools.combinations 遍历所有可能的子集，组合总数为 2^k。对于每个组合需要遍历其包含的边，因此总复杂度为指数级 $O(E \cdot 2^{E-V+1})$。
- 空间复杂度：需要存储所有生成的环路，最坏情况下也是指数级。

（五） 实验结果

列出对于给定的输入所产生的输出结果。若可能，测试随输入规模的增长所用算法的实际运行时间的变化。

1、$A=\begin{bmatrix}0 & 1 & 1 & 1\\ 1 & 0 & 0 & 1\\ 1 & 0 & 0 & 1\\ 1 & 1 & 1 & 0\end{bmatrix}$

输出结果：

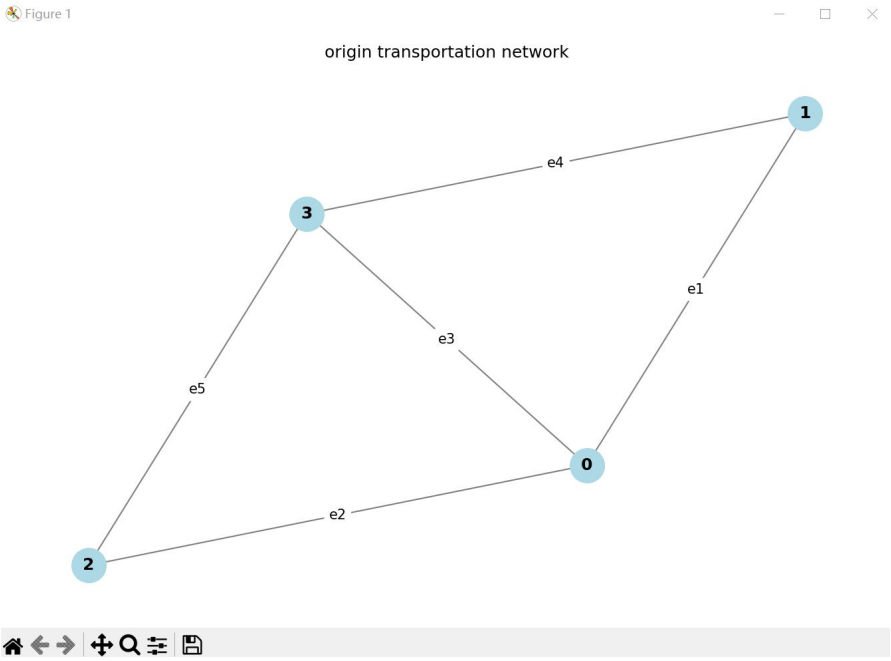
```
请输入顶点数 n：4
请逐行输入邻接矩阵（共 4 行，每行 4 个数字，空格分隔）：
第 1 行：0 1 1 1
第 2 行：1 0 0 1
第 3 行：1 0 0 1
第 4 行：1 1 1 0
原始交通网络邻接矩阵（4x4）：
[[0 1 1 1]
 [1 0 0 1]
 [1 0 0 1]
 [1 1 1 0]]
标定边：
  e1: (0, 1)
  e2: (0, 2)
  e3: (0, 3)
  e4: (1, 3)
  e5: (2, 3)

========== 生成树个数 ==========
关联矩阵为（4x5）：
[[ 1  1  1  0  0]
 [-1  0  0  1  0]
 [ 0 -1  0  0  1]
 [ 0  0 -1 -1 -1]]
生成树个数为 : 8

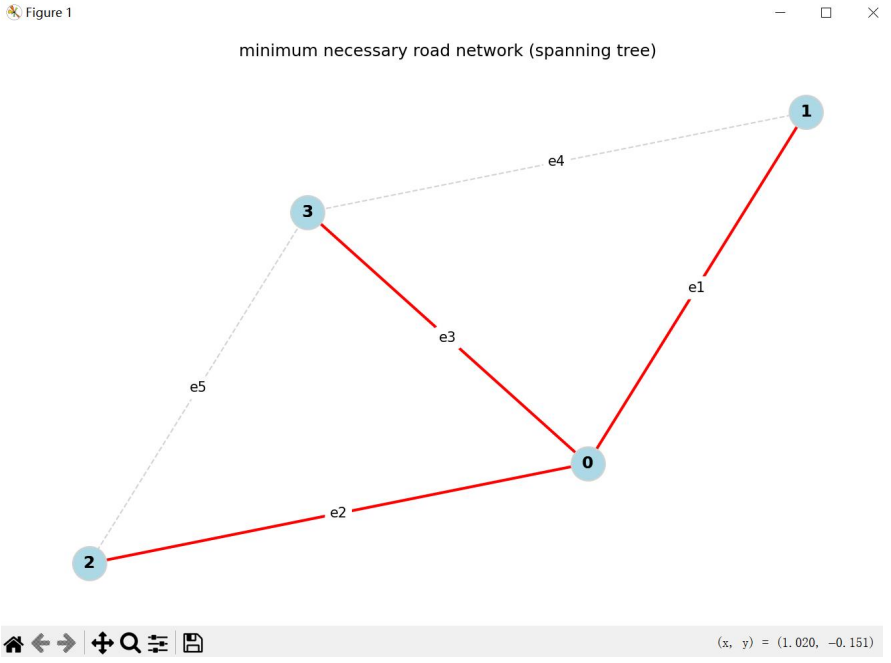
========== 最小必要路网（生成树） ==========
构建的 '最小必要路网'（生成树）边集为：
{ e1, e2, e3 }
最小绕行单元库（基本回路系统）- 共 2 个：
{e1e3e4, e2e3e5}

========== 所有绕行方案（环路空间） ==========
所有绕行方案（环路空间）- 共 4 个：
{Φ, e1e3e4, e2e3e5, e1e2e4e5}
```

原始交通网络：



生成树：



2、$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$

输出结果：

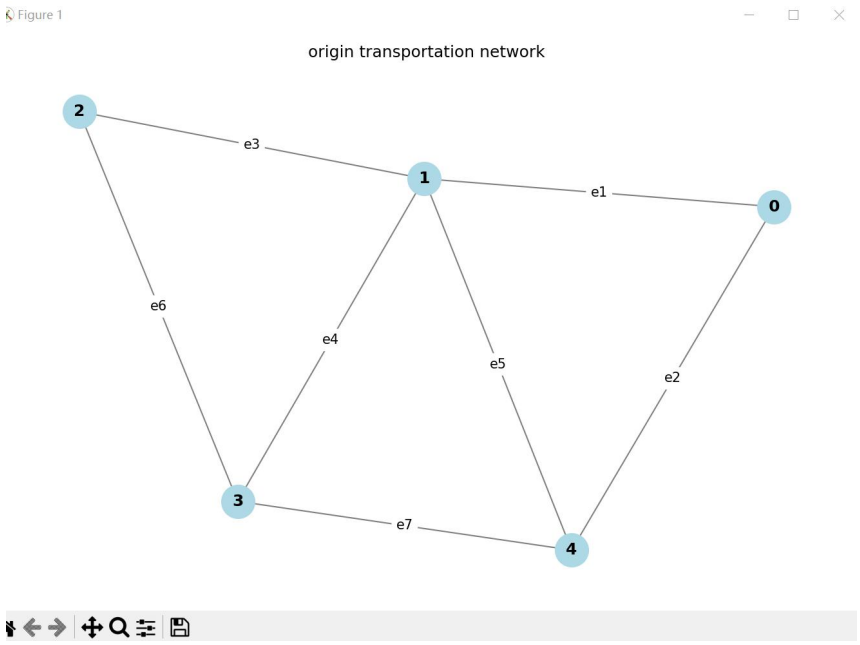
```
D:\编译器\python.exe "D:\yyr\study\数学\离散2\lab3\离散 实验3.py"
========== 城市交通网络分析 ==========
请输入顶点数 n：5
请逐行输入邻接矩阵（共 5 行，每行 5 个数字，空格分隔）：
第 1 行: 0 1 0 0 1
第 2 行: 1 0 1 1 1
第 3 行: 0 1 0 1 0
第 4 行: 0 1 1 0 1
第 5 行: 1 1 0 1 0
原始交通网络邻接矩阵 (5x5):
[[0 1 0 0 1]
 [1 0 1 1 1]
 [0 1 0 1 0]
 [0 1 1 0 1]
 [1 1 0 1 0]]
标定边：
  e1: (0, 1)
  e2: (0, 4)
  e3: (1, 2)
  e4: (1, 3)
  e5: (1, 4)
  e6: (2, 3)
  e7: (3, 4)

========== 生成树个数 ==========
关联矩阵为 (5x7):
[[ 1  1  0  0  0  0  0]
 [-1  0  1  1  1  0  0]
 [ 0  0 -1  0  0  1  0]
 [ 0  0  0 -1  0 -1  1]
 [ 0 -1  0  0 -1  0 -1]]
生成树个数为 : 21
```

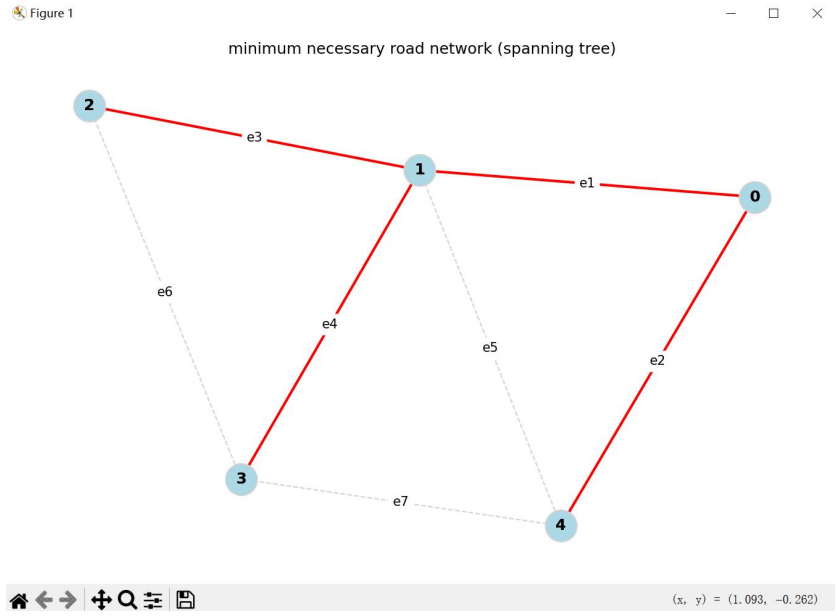
```
========== 最小必要路网（生成树） ==========
构建的 '最小必要路网' (生成树) 边集为:
{ e1, e2, e3, e4 }
最小绕行单元库 (基本回路系统) - 共 3 个:
{e1e2e5, e3e4e6, e1e2e4e7}

========== 所有绕行方案（环路空间） ==========
所有绕行方案 (环路空间) - 共 8 个:
{Φ, e1e2e5, e3e4e6, e1e2e4e7, e1e2e3e4e5e6, e4e5e7, e1e2e3e6e7, e3e5e6e7}
```

原始交通网路：



生成树：



（六） 实验总结

通过本次实验，我深刻体会到了离散数学图论知识在解决实际工程问题中的核心作用。实验将枯燥的“无向连通图”抽象为生动的“城市交通网络”，将“生成树”转化为规划中至关重要的“最小必要路网”。我理解了生成树的数量不仅仅是一个数学结果，它更直接反映了交通网络的拓扑稳健性 — 生成树越多，意味着网络在遭遇局部路段故障时的替代方案越多，系统的抗毁性越

强。

在生成环路空间时，对基本回路进行子集组合的复杂度为 $O(2^k)$。这让我深刻认识到，虽然穷举法和递归定义在数学上严谨且优美，但在处理大规模数据（NP-Hard 类问题特征）时往往不再适用，工程实践中需要权衡精度与效率，或寻求如高斯消元法（$O(N^3)$）等更优解。

本次实验通过编程实现矩阵树定理，我掌握了如何通过关联矩阵及其转置的乘积来计算生成树个数。这让我看到，图的结构信息（连通性）可以被精确地编码在矩阵的代数性质（行列式）中，这种跨学科的数学工具在计算机科学中具有极高的美学和实用价值。

实验最后的可视化模块不仅是结果的展示，更是调试的利器。利用 NetworkX 和 Matplotlib，我实现了在图上直接标注边编号（e1,e2...）。这种直观的反馈帮助我迅速验证了“基本回路”是否真的构成了闭环，以及“最小必要路网”是否确实覆盖了所有顶点且无环。将抽象的邻接矩阵转化为可视化的网络图，极大地提升了对算法运行结果正确性的信心。